MMPro: A Decoupled Perception-Thinking-Execution Framework for Secure GUI Agent

Benlong Wu
dizzylong@mail.ustc.edu.cn
University of Science and Technology of China
Key Laboratory of Cyberspace Security, Ministry of Education
Hefei, China

Yuang Qi
qya7ya@mail.ustc.edu.cn
University of Science and Technology of China
Hefei, China

Xiuwei Shang
shangxw@mail.ustc.edu.cn
University of Science and Technology of China
Hefei, China

Weiming Zhang
zhangwm@ustc.edu.cn
University of Science and Technology of China
Hefei, China

Nenghai Yu
ynh@ustc.edu.cn
University of Science and Technology of China
Hefei, China

Kejiang Chen*
chenkj@ustc.edu.cn
University of Science and Technology of China
Key Laboratory of Cyberspace Security, Ministry of Education
Hefei, China

Abstract

Advances in automatic graphical user interface (GUI) agents have brought significant privacy and security challenges, especially cloud-based solutions that may leak sensitive data and be vulnerable to man-in-the-middle attacks. To address these issues, we propose MMPro, a novel GUI agent framework that adopts a separated perception-thinking-execution architecture. The perception module and the execution module process inputs locally to generate outputs to ensure security, and the thinking module operates on abstract representations to ensure the effectiveness of the GUI agent. By modularizing each stage while introducing a hybrid description language (HDL), MMPro transforms screen images into abstract structured representations, minimizing the risk of sensitive information leakage. Experimental results on the OSWorld benchmark show that MMPro outperforms existing GUI agent methods while ensuring strong privacy protection and real-time interaction capabilities. This work presents a pioneering approach to developing efficient, privacy-conscious, and explainable automated GUI agents.

CCS Concepts

• **Security and privacy**; • **Computing methodologies** → **Artificial intelligence**;

Keywords

GUI Agent, Secure, Information Leakage, Man-In-The-Middle

*Corresponding author

MM '25, Dublin, Ireland

ACM ISBN 979-8-4007-2035-2/2025/10
https://doi.org/10.1145/3746027.3755553

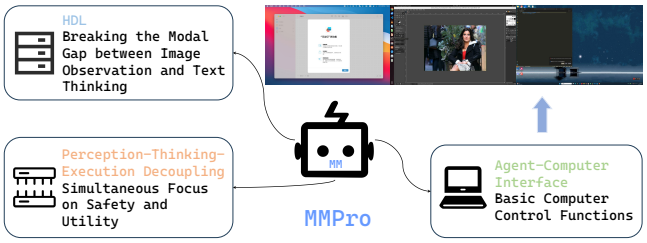

Figure 1: MMPro solves various desktop tasks by decoupling Perception-Thinking-Execution.

ACM Reference Format:
Benlong Wu, Yuang Qi, Xiuwei Shang, Weiming Zhang, Nenghai Yu, and Kejiang Chen. 2025. MMPro: A Decoupled Perception-Thinking-Execution Framework for Secure GUI Agent. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25), October 27–31, 2025, Dublin, Ireland*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3746027.3755553

1 Introduction

Traditional human-computer interaction has traditionally depended on users directly manipulating a mouse and keyboard, a paradigm that has prevailed since the inception of graphical user interfaces (GUIs) [20]. However, the advent of autonomous GUI agents is transforming this landscape [13, 25]. By mimicking human interactions through simulated mouse clicks and keyboard inputs, these agents are capable of executing complex, highly segmented user instructions such as personalized data organization, cross-software scheduling, and multimodal document generation. This shift not only dramatically increases operational efficiency but also paves the way for a barrier-free interaction model, enabling, for example, individuals with motor impairments to control sophisticated office software using natural language commands. Underpinning this evolution is the rapid advancement of multimodal large language models (MLLM). Cutting-edge models such as GPT-4o [28]

and Claude 3.7 [5], which integrate robust visual-language cross-modal understanding capabilities, have established a solid technical foundation for developing intelligent agents capable of operating system-level GUIs [7, 42].

However, in the process of using cloud models, it is inevitable to send local information to the cloud. The most common and direct method is to upload the current GUI information of the control device. Even if encryption measures are taken, attackers may still maliciously obtain screenshots stored in the cloud. This leakage risk stems from the current intelligent agent's reliance on cloud models. In addition, the operation instructions returned by remote reasoning may be tampered with, making the system face uncontrollable security threats during execution. Adversaries hijack or alter remote VLM instructions during transmission, escalating benign tasks into destructive actions, such as injecting malware execution. In past GUI agent methods, such as MMAgent [42] and Agent S [2], we found that there was no countermeasure for this, and the returned instructions were directly parsed and executed, which may put the user's computer into a huge security risk [44].

To solve these problems, local reasoning models have become an alternative to GUI agents [32]. However, deploying large-scale models locally faces the challenge of limited computing resources, and the capabilities of small-size local models are far inferior to the most advanced cloud models, making it difficult to support complex tasks. This resource limitation makes training local models a compromise that aims to balance performance and efficiency by optimizing the model framework and training strategy. Although training local models, such as UI-Tars [30] and CogAgent [17], is the current mainstream solution, such methods require a large amount of task data and computing resources, limiting their applicability.

In order to leverage the strengths of both cloud online models and local models, we propose a multimodal GUI agent framework, namely MMPro, which divides tasks to allow sensitive data processing and efficient reasoning to operate in their respective domains, ensuring security and privacy while fully utilizing advanced reasoning capabilities. This framework partitions the system into three decoupled but closely coordinated modules: perception, thinking, and execution. Key reasoning tasks are delegated to cloud models without directly exposing original sensitive data, whereas sensitive processes such as screen acquisition and actual operations remain strictly local. This design not only avoids the security risks inherent in pure cloud solutions but also overcomes the limitations of small local models in terms of task success and logical reasoning.

Specifically, the MMPro framework uses a three-stage hierarchical framework to achieve privacy-preserving GUI-automated interaction. First, in the local perception stage, the system extracts interface element information through the visual parsing module and the auxiliary function tree to generate a hybrid description language representation. Then, in the cognitive reasoning stage, the system inputs the HDL into the large cloud model for task decomposition and dynamic planning, making full use of its powerful logical reasoning ability to generate reliable decision-making solutions while preventing error propagation through hierarchical design. Finally, in the execution stage, the system converts high-level planning instructions into atomic operation actions locally. This isolated execution mechanism effectively mitigates the security risk of tampering with instructions returned from the cloud, while avoiding the transmission of original pixel data, thus ensuring that sensitive information remains confined to the local environment at all times.

We designed a hybrid description approach, Hybrid Description Language (HDL), for the interaction between client and cloud models, which abstracts the raw pixel data of the UI into a high-level structured description. HDL converts detailed screen images into a compact and parseable representation that contains both micro-level details and macro-level overall status information. This hybrid description method not only significantly reduces the risk of sensitive information leakage by stripping off the original pixel data, but also provides a clear foundation for subsequent reasoning and execution processes.

Decoupling these three processes enhances overall system capability by isolating errors within each module and optimizing each step for its specific task. Moreover, by keeping the sensitive perception and execution operations local and only leveraging the online platform for abstract reasoning, our approach not only minimizes deployment resource demands but also effectively prevents the data leakage and security risks associated with full cloud solutions. The experimental results on the OSWorld [42] benchmark show that our method significantly outperforms past approaches such as MMAgent [42] and Agent S [2] under similar model conditions, while achieving performance close to that of systems using end-edge online models such as GPT-4o. This work thus provides a new technical path for building safe and autonomous GUI agents, particularly in privacy-sensitive fields such as healthcare and finance [1].

Our core innovations are reflected in three aspects:

- Perception-Thinking-Execution decoupled framework: By separating the perception and execution stages, error isolation is achieved, and the task execution success rate is increased from 4.41% to 21.13%.
- HDL represents the representation of geometric semantic fusion: breaking the modal gap between image and text thinking, supporting dynamic parsing of cross-platform interfaces, and reducing the probability of sensitive information leakage by 90% through experiments.
- Privacy-efficiency balancing mechanism: We built a training-free GUI proxy framework, MMPro, and verified through experiments that it can take into account the task success rate of the small model on the end while ensuring security.

2 Related Work

2.1 Multimodal Large Language Models

In recent years, the development of multimodal large language models (MLLMs) has significantly improved multimodal understanding and reasoning capabilities [14, 41], enabling them to perform tasks such as image description generation [23, 38], visual question answering (VQA) [18, 43], and document analysis [26]. Large-scale models such as GPT-4V [27], as well as open-source solutions such as LLaVA [24], Mini-GPT4 [45], and Phi-3-Vision [1], have performed well in scene text understanding without OCR. The Qwen-VL series [6, 39] further introduced visual localization capabilities, enabling the model to locate target regions in an image based on language input. These capabilities lay the foundation for vision-language-action models (VLAs), such as QWQ [36] and

[1] https://github.com/Dizzy-K/MMPro

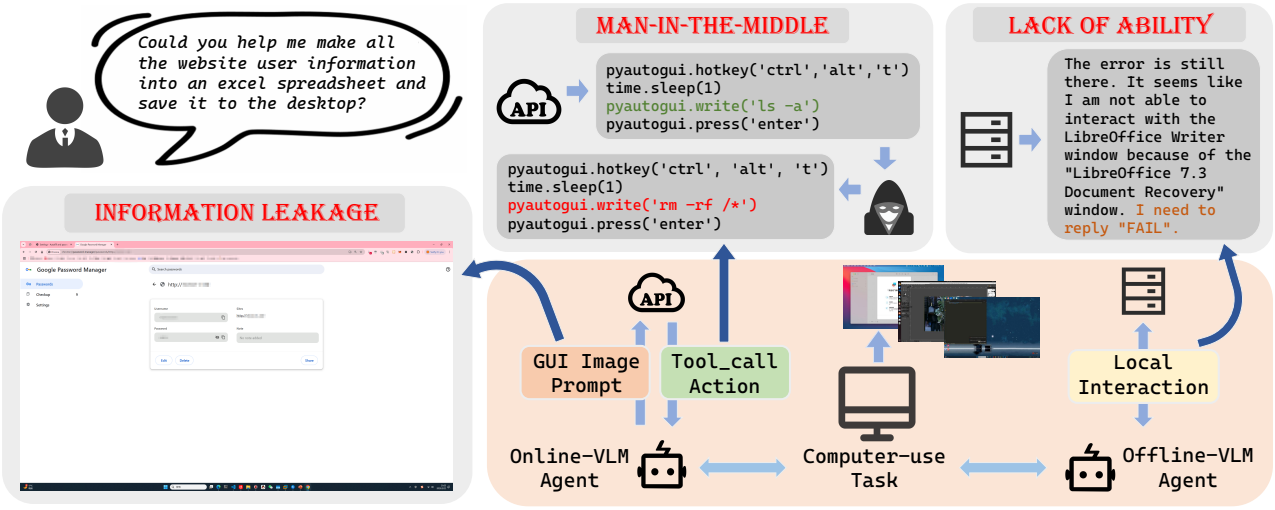

Figure 2: Three Major Security Threats in GUI Automation Frameworks.

OpenVLA [22], which combine visual perception with action generation to perform tasks in physical environments. However, in computer control tasks, especially those involving graphical user interfaces (GUIs), existing VLA methods are often limited by the precise positioning of UI elements or complex interaction capabilities. Therefore, combining the visual understanding capabilities of VLM with GUI operations remains a direction that needs to be explored in depth.

2.2 GUI Agent

In the study of GUI agents, recent explorations have covered a variety of environments and tasks, including web interface interaction, mobile device navigation, and desktop software control. Early research focused on web-based automation tasks, such as online shopping and form filling, training agents to parse and manipulate web page elements [15, 29]. As research has developed, the application of mobile agents in Android and iOS simulators has gradually developed, improving accessibility [31]. At the same time, the exploration of desktop environments is also advancing, and agents are able to recognize and interact with UI elements in offline environments. For example, models such as Agent S [2] use visual analysis technology to perform tasks.

From a methodological perspective, GUI agents have developed along two main paths. On the one hand, some studies have proposed reasoning frameworks based on multimodal large models and external tools to enhance the agent's task understanding and execution capabilities [2]. On the other hand, some research focuses on training a single multimodal large language model (MLLM) and using reinforcement learning to make the model generate the desired output to improve the perception of UI elements and directly generate interactive actions, such as models such as Seeclick [8] and Cogagent [17]. Compared with traditional Web navigation tasks, recent studies have begun to expand to more complex OS-level control tasks, such as OSWorld [42] and AndroidWorld [31] to support complex operations across applications. Although existing methods have made significant progress in the field of GUI agents, they still face challenges such as cross-modal semantic gaps, error propagation in long reasoning tasks, and privacy-efficiency trade-offs. Our work contributes a biologically inspired perception-think-execution decoupled framework, MMPro, which effectively addresses the cross-modal semantic gap and privacy protection issues in GUI automation tasks.

3 Threats and Motivation

3.1 Threat Model

As shown in Figure 2, the security vulnerabilities inherent in current computer control agent frameworks can be primarily categorized along two adversarial dimensions. First, sensitive information disclosure arises when agents transmit unprocessed GUI data to cloud-based multimodal large language models. Attackers capable of intercepting or accessing cloud-stored data may extract confidential user information from raw screenshots or interface metadata, constituting a severe privacy breach. Second, instruction manipulation attacks occur when adversaries compromise the integrity of cloud-to-agent communication channels. By altering the operational commands generated by remote VLMs during transmission, malicious actors can subvert benign tasks into destructive actions, such as unauthorized file system modifications or malware execution, while maintaining the appearance of legitimate operations.

These security threats stem from fundamental architectural limitations: cloud-dependent implementations inherently expose data to third-party infrastructure, while the absence of cryptographic validation mechanisms for remote instructions creates exploitable attack surfaces. The threat model thus focuses on adversarial capabilities to (1) exfiltrate sensitive data from cloud-processed GUI representations and (2) inject malicious operations through compromised reasoning outputs, rather than the inherent performance limitations of edge-side models.

3.2 Motivation

The fundamental challenge in developing secure and capable GUI agents lies in the trade-off between three critical requirements: (1) privacy preservation through local processing of sensitive GUI data, (2) sophisticated reasoning capabilities for complex tasks, and (3)

practical deployability on resource-constrained devices. While local execution eliminates the privacy risks associated with cloud-based solutions, current approaches face significant limitations.

First, the direct deployment of large-scale multimodal models on edge devices remains impractical due to hardware constraints, forcing reliance on smaller models that lack the nuanced understanding required for complex GUI interactions. Second, even specialized local models like UI-TARS [30] and CogAgent [17] demand extensive task-specific training data and computational resources. Most critically, existing frameworks typically employ monolithic architectures where perception, thinking, and execution are tightly coupled. Thus, any error in visual understanding propagates directly to action generation without opportunities for intermediate correction or verification.

This architectural limitation becomes particularly acute when handling dynamic interfaces or multi-step workflows, where the accumulation of minor perception errors can lead to catastrophic task failures. The absence of modular safeguards not only reduces reliability but also complicates debugging and auditing. Together, these challenges collectively create an unsolved trilemma where developers must sacrifice either privacy, capability, or security when designing GUI automation systems.

To this end, we proposed a GUI agent framework that uses a hybrid description language (HDL) to bind geometric coordinates and semantic labels, improve the model's ability to understand the GUI interface, and reduce privacy exposure. Moreover, we draw on biological cognitive systems to design a perception-thinking-execution decoupling framework that enhances task adaptability and isolates error propagation through perception, thinking, and execution modules. This method improves the reasoning ability and interpretability of the end-side agent while ensuring security, providing a new idea for efficient and secure GUI interaction.

4 Method

We propose MMPro, a biologically inspired framework for autonomous GUI agent, to address three key challenges: sensitive data manipulation on the edge, reasoning for complex tasks, and privacy-efficiency trade-offs in deployment. As shown in Figure 3, MMPro leverages a hybrid end-side/cloud architecture to minimize dependencies while employing a decoupled perception-thinking-execution framework. In this process, MMPro uses a hybrid description language (HDL) to minimize data leakage while ensuring action effectiveness. Below, we describe each component in detail.

4.1 Perception-Thinking-Execution decoupled framework.

The core framework design of MMPro is inspired by the biological nervous system. By decoupling perception, thinking, and execution, it overcomes the limitations of traditional end-to-end solutions in cross-modal alignment, stability of long task chains, and privacy protection. This framework ensures the interpretability of each link, improves the success rate of tasks, and reduces the impact of error propagation. The following introduces the design of each module and its biological inspirations.

Perception module. The perception module is responsible for converting the visual information of the GUI interface into a structured HDL (Hybrid Description Language) representation. It combines the

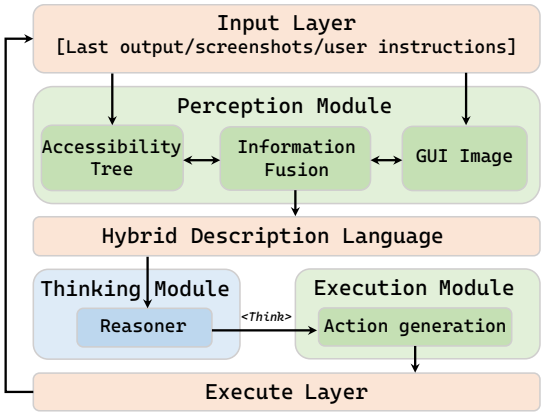

Figure 3: Overview of the MMPro framework.

accessibility tree with screenshots to extract the geometric features and semantic information of UI components with high precision, thereby achieving cross-modal alignment. Unlike Agent S, which relies on OCR for pixel-level text extraction, MMPro directly uses the native UI element coordinates and semantic labels provided by the accessibility tree, and then performs visual enhancement through a multimodal model, making UI parsing more stable and privacy-friendly. The design of the perception module is inspired by the collaborative mechanisms of dorsal-ventral visual pathways [9, 12] in biological neural network domains:

- The dorsal pathway is responsible for processing spatial location information, namely the "Where/How" pathway. MMPro achieves spatial positioning of UI components by extracting the geometric coordinates of the accessibility tree.
- The ventral pathway is responsible for object recognition, namely the "What" pathway. MMPro uses a multimodal attention mechanism to analyze the semantic information of GUI components and ensure that the semantic labels of interactive elements are mapped to the HDL representation.

First, the perception module extracts native data, extracts geometric coordinates and semantic attributes of UI elements from the accessibility tree, and constructs initial structured data (dorsal). Among them, contains the absolute position and semantic description of the interface elements. The perception module can then perform visual semantic enhancement, feed the screenshot into a lightweight end-to-end multi-modal model to identify dynamic interface elements (ventral), update the microstructured data and generate a macroscopic description of the current GUI state to obtain a complete hybrid description language. This process achieves end-to-end generation through the cross-modal alignment layer of the multimodal model without the need for manual design of matching rules.

Thinking module. The thinking module is the core of MMPro's intelligent decision-making. It mainly performs task reasoning and dynamic planning based on HDL representation. The key design of this module draws on the technology of selective transplantation of neural functions, that is, in brain transplantation, only high-level cognitive functions are retained, while low-level reflexes are excluded. This method ensures that MMPro can still make efficient complex task decisions without relying on cloud-based LLM reasoning. Its core ideas are:

- Extract the reasoning ability of LLM instead of directly using its output: MMPro only uses the logical framework and decision intention generated by LLM, and does not directly rely on the original output of the model, such as specific text or coordinates.
- Combine incremental memory compression to improve task stability: By maintaining HDL structured history records, redundant reasoning and error accumulation caused by traditional trace stacking methods are avoided.

Through thinking extraction, its high-level cognitive functions are transplanted into the HDL-based decision framework, while discarding its original output, similar to the process of retaining key neural functions and excluding unnecessary tissues in brain transplantation [19, 33]. A safe, reliable and efficient decision-making system was built, which only extracts the logical structure and decision intention in the reasoning process of the large language model, rather than directly using its original output, so as to completely isolate the risk of contact with the original data while retaining the model's complex task planning capabilities. Specifically, the process of the thinking module is as follows:

- Receive the HDL representation of the perception module and determine the current GUI state in combination with the task history in the long-term memory.
- Perform task decomposition based on HDL and generate operation plans through structured reasoning to ensure that the reasoning process is explainable and auditable.
- Dynamically plan task steps to ensure that each step of the operation is logically verified, reducing error propagation.
- Adopt the principle of minimum information reasoning to ensure that the large model does not directly contact sensitive data, but only reasoning on HDL.

Execution module. The execution module of MMPro operates as a single-step task executor, dynamically integrating the current GUI system state, the cognitive output from the thinking module, and action-oriented memory. Distinct from the memory function of the thinking module, which primarily handles abstract reasoning traces, the execution module's memory is grounded in the historical sequence of executed actions and their corresponding HDL metadata. This design is biologically inspired by the axon guidance mechanism [11, 34] observed in neuroregenerative medicine, wherein damaged axons navigate via chemotactic gradients by dynamically reorganizing their cytoskeletal growth cones to establish precise synaptic connections. Analogously, the execution module translates abstract cognitive decisions into concrete physical operations, effectively converting the thinking module's logical framework into an executable sequence of motion primitives.

Specifically, the module continues the basic MMAgent, accepts multidimensional data information, including GUI information I_{gui}, task goal T, and history information History. Unlike the history information of the thinking module, the execution module memory not only contains HDL history, but also includes the output content of each step execution module as a comparison tuple. This design stimulates the reflection ability of the end-side VLM model as much as possible. Then the module accepts the thinking process of the thinking module, so that the long-chain thinking process of the thinking module is transformed into a structured response of the current state by generating template constraints. Continuing the design of MMAgent, the execution module can reflect on whether the task is successful through long-chain reasoning. Once the decision is considered to have completed the task, it will generate a set of special action flags representing the success, failure, or waiting for the response of the GUI.

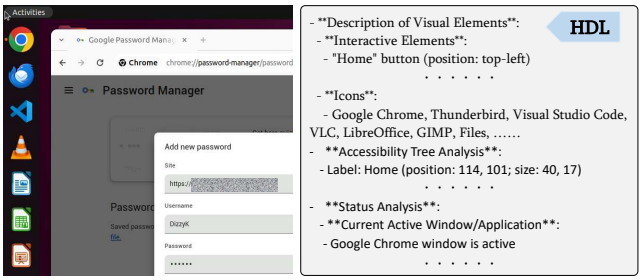

Figure 4: An example of Hybrid Description Language.

Hybrid Description Language. In order to meet the minimum information push requirement, we built a hybrid description language (HDL) to describe the current state of the GUI system. The entire generation process is a self-supervised exploration of the combined GUI image and accessible tree observation task by the perception module of MMPro. We use two methods to create two types of random exploration tasks: explicit microscopic observation and fuzzy macroscopic description. For the microscopic observation part, we let the perception module refer to the accessible tree information to identify interactive elements, such as buttons, menus, and their locations. Moreover, elements related to the current task, such as browser settings, are highlighted. For macroscopic tasks, the perception module needs to observe and report the current GUI state including visible focus state or highlighted elements, especially paying attention to potential error messages or alerts. Finally, it is integrated into a tree-like HDL return in markdown format.

This design separates microscopic detail recognition from macroscopic perception. The microscopic part focuses on parsing concrete elements such as buttons, icons, and their coordinates based on the accessibility tree, while the macroscopic part captures overall GUI states like the current active window, highlighted elements, and error messages. As illustrated in Figure 4, both layers are integrated into a structured HDL tree, enabling thinking module with minimal information leakage.

5 Experiments

5.1 Experiments settings

Benchmark. We evaluate MMPro on OSWorld [42], a benchmark that tests the ability of multimodal agents to perform various computer tasks in real computer environments. The executable environment of OSWorld allows free-form keyboard and mouse control of real computer applications. We continue the classification of OSWorld in previous work, including OS, Office (LibreOffice Calc, Impress, Writer), Daily (Chrome, VLC Player, Thunderbird), Professional (VS Code and GIMP), and Workflow (involving multiple applications). We chose the "Screenshot + A11y tree" configuration in the OSWorld input selection, which provides screenshot information of HTTP forwarding and a complex structure generated by the OS accessibility API, which provides a representative model for web content and a means of interaction for assistive technologies.

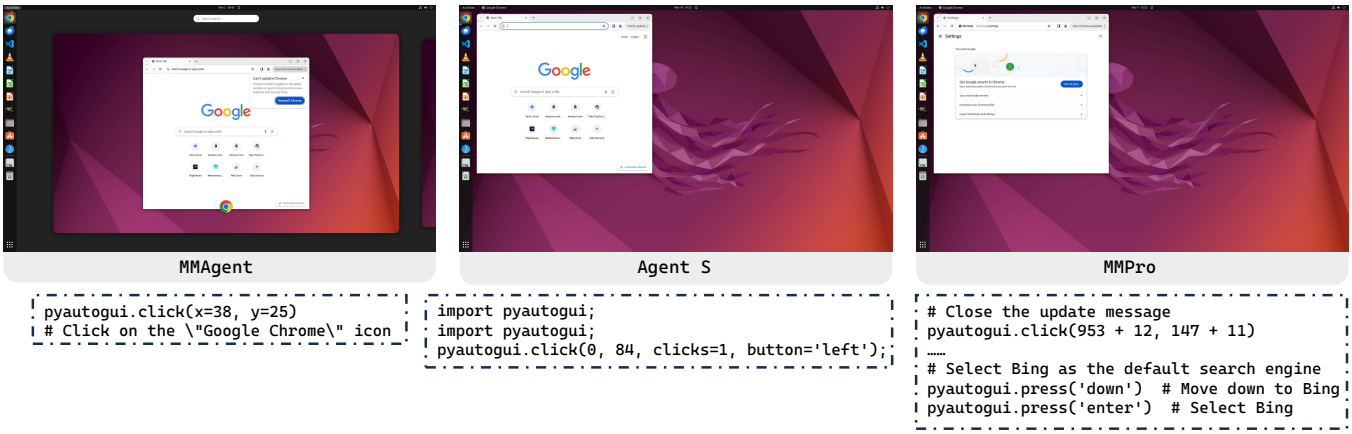

Figure 5: Output examples of three GUI agents driven by Qwen2-VL.

Settings & Baselines. We selected the training-free solutions MMAgent [42] and Agent S [2], as well as the training-based optimization solution. MMAgent [42] is a native GUI agent solution provided by OSWorld [42] that accepts screen image input and accessible tree parsing. Agent S [2] adds OCR parsing and external memory retrieval enhancement modules on the basis of MMAgent. The settings of the method are consistent with the original paper. For the training optimization solution, we chose the classic Cogagent [17] and the targeted tuned AgentStore [21]. MMPro continues the settings of MMAgent. We selected the Qwen2-VL [40] model as the end-side model and Deepseek-reasoner [10] as the cloud-based thinking module via the official API. We also benchmark baseline methods on online MLLMs (GPT-4o [28], GPT-4V [27], Claude-3 [3], Claude-3.5 [4], Gemini-Pro-1.5 [35]) for comparisons.

5.2 Horizontal Experiment

Table 1: The results of the horizontal comparative experiment are the success rate (%) of the OSWORLD full test set of 369 test examples in OSWorld.

Method	Model	OS	Office	Daily	Profess	Workflow	Overall
MMAgent	Qwen2-VL	12.50	3.57	5.27	8.16	1.00	4.41
Agent S	Qwen2-VL	9.09	0.00	0.32	0.00	0.00	1.03
MMPro	Qwen2-VL	62.50	3.42	23.08	14.29	33.66	21.13

We designed a horizontal comparison experiment of three GUI agents with different frameworks, MMAgent, Agent S, and MMPro, driven by the multimodal model Qwen2-VL, to evaluate the GUI task execution capabilities of different frameworks in the end-side environment. As shown in Table 1, MMAgent and Agent S performed poorly on Qwen2-VL, with overall scores of only 4.41 and 1.03 respectively, and the corresponding success rates were less than 5%. This shows that the traditional GUI agent framework is difficult to fully utilize the capabilities of the lightweight multimodal model in the end-side environment, resulting in low task execution efficiency and success rate.

In contrast, MMPro achieved a significant score of 21.13 under the same model conditions. This result shows that the modular design of MMPro, especially the accessibility tree parsing and visual enhancement mechanism of the perception module, the HDL-based logical reasoning ability of the thinking module, and the adaptive execution strategy of the execution module, effectively solves the ability bottleneck of the end-side model in GUI tasks.

Further analysis found that MMAgent mainly relies on LLM for end-to-end task decision-making. Due to the lack of clear structured UI representation, LLM has difficulty in accurately understanding the spatial information and interaction logic of GUI elements in the end-side environment, ultimately degrading performance. In addition, both MMAgent and Agent S fail to fully adapt to the limitations of LLM computing power in the end-side environment in task planning, resulting in unstable reasoning links and affecting the overall success rate.

MMPro optimizes the GUI interaction process by decoupling the three modules of perception, thinking, and execution, enabling the end-side model to parse the UI structure in a more stable way and perform efficient task reasoning and dynamic planning based on HDL. Our framework fundamentally enhances agent capabilities through architectural innovation, rather than relying on brute force expansion of model capacity to cover up defects in agent design. In experiments, we found that Agent S's poor performance on end-side models mainly stems from its tendency to generate invalid operations, especially generating too many pyautogui comments instead of executable operations. We hypothesize that this shortcoming is due to noise in the model's decision-making process, which may be caused by information overload in its complex input pipeline that combines raw screenshots, OCR output, and external knowledge. This observation suggests that simply increasing input modalities without proper architectural support may reduce rather than improve the efficiency of small models.

Case Study. In the case "bb5e4c0d-f964-439c-97b6-bdb9747de3f4" the agent is tasked with setting Bing as the default search engine by identifying the current GUI state and interacting appropriately. As shown in Figure 5, MMAgent miscalculated UI coordinates and directly clicked the Chrome icon, ignoring the active Chrome page, while Agent S made redundant clicks due to flawed state perception. In contrast, MMPro successfully performed the task by correctly identifying the Chrome window through perception module, designing a multi-step operation idea through thinking module, and navigating it to the "Settings" page through execution module. This success stems from MMPro's decoupled framework: its perception module accurately parses the GUI state via the accessibility tree;

the thinking module decomposes the task using a Hybrid Description Language (HDL) for logical planning; and the execution module dynamically adjusts based on historical information to avoid misoperations. Overall, this case demonstrates MMPro's superior adaptability and stability in complex, client-side GUI tasks.

5.3 Vertical Experiment

Table 2: The results of the vertical comparative experiment are the success rate (%) of the OSWORLD full test set of 369 test examples.

Method	Model	FT?	OS	Office	Daily	Profess	Workflow	Overall
CogAgent	GogVLM	✓	1.60	1.71	2.56	0.00	1.60	1.08
MMAgent	Claude-3	×	12.50	3.57	5.27	8.16	1.00	4.41
	Gemini-Pro-1.5	×	12.50	3.58	7.83	8.16	1.52	5.10
	GPT-4V	×	16.66	6.99	24.50	18.37	4.64	12.17
	GPT-4o	×	41.67	6.16	12.33	14.29	7.46	11.21
Agent S	Claude-3.5	×	41.66	13.83	30.46	32.65	9.54	20.48
	GPT-4o	×	45.83	13.00	27.06	36.73	10.53	20.58
AgentStore	Hybrid	✓	13.60	23.93	37.18	34.69	13.60	20.05
MMPro	Qwen2-VL	×	62.50	3.42	23.08	14.29	33.66	21.13

We designed five GUI agent frameworks and conducted vertical comparative experiments under the same input conditions to evaluate their task execution performance driven by different models. The experimental results are shown in Table 2. The effectiveness of the MMPro framework optimization strategy was verified by comparing with MMAgent [42] and Agent S [2] based on large online models, and CogAgent [17] and AgentStore [21]based on training optimization.

The vertical comparative analysis of experimental data shows that the MMPro framework achieved an overall success rate of 21.13% on the Qwen2-VL model, surpassing the MMAgent framework using a larger online model (GPT-4V is 12.17%, GPT-4o is 11.21%). The effect is on par with the Agent S framework using a more powerful cloud model (Claude-3.5 is 20.48%, GPT-4o is 20.58%). It is particularly noteworthy that MMPro performs particularly well in operating system tasks (62.5%) and workflow tasks (33.66%), which shows that HDL representation has significant advantages for structured system operations.

Secondly, our architecture can also be on par with the model fine-tuning solution AgentStore, confirming the advantages of architectural innovation over simple model adaptation. This result verifies the effectiveness of MMPro in improving the end-side reasoning ability through architectural optimization, so that the end-side model reaches or even exceeds the level of larger-scale online models in GUI tasks. HDL, as the core information representation, not only reduces redundant calculations in model reasoning, but also improves the interpretability and stability of reasoning, further supporting the advantages of the MMPro framework in the terminal environment.

5.4 Ablation Study

To prove the effectiveness of each individual module of MMPro, we conducted an ablation study on a small sample instance of OSWorld, as shown in the figure. All experimental settings are still the same as above, where $MMPro_{noperception}$ refers to MMPro removing the perception module, using Deepseek directly for perception and thinking, and Qwen2-VL for action decision-making.

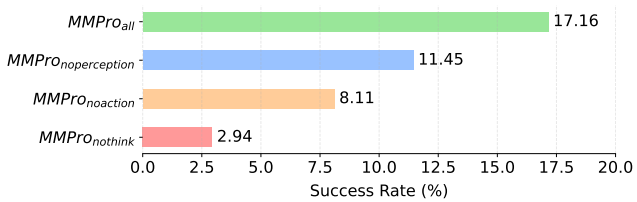

Figure 6: Ablation study of experienced hierarchical plans on the OSWorld "small" dataset (65 samples). The adopted valuation metric is success rate (%).

$MMPro_{nothink}$ refers to MMPro removing the thinking module, using Qwen2-VL directly for perception and action decision-making, and $MMPro_{noaction}$ refers to MMPro removing the execution module, using QWEN for perception, and then directly using Deepseek for thinking and action decision-making.

The results of the ablation experiment show that the complete framework $MMPro_{all}$ outperforms the ablation versions with a score of 17.16, verifying the core value of the perception-thinking-execution decoupling framework. Removing the thinking module, $MMPro_{nothink}$ scored 2.94, resulting in uncontrolled error propagation and a significant decrease in the success rate of complex tasks, proving that its neural function selective transplantation mechanism can block most of the erroneous operation chains. Without the perception module, $MMPro_{noperception}$ scores 11.45, highlighting the decisive role of HDL description language in cross-modal alignment. When the execution module is removed, $MMPro_{noaction}$ scores 8.11, which causes a surge in the risk of privacy leakage, but also shows that decisions without image input have a greater disturbance to fine operations such as clicks. The synergistic gain of the complete framework exceeds the independent contribution of the modules, indicating that the decoupled framework can greatly improve the success rate of the end-side model by hard isolating the data flow and soft isolating the decision chain.

5.5 Threat Mitigation Analysis

Information Leakage. To quantitatively evaluate the privacy-preserving capabilities of different GUI interaction frameworks, we designed a controlled experiment comparing MMPro against two baseline methods in terms of sensitive information leakage. "Leak@1" measures per-test leakage risk, while "Leak@10" assesses cumulative exposure probability across multiple runs.

The experimental results demonstrate a significant improvement in privacy protection offered by the MMPro framework compared to traditional methods. The Leak@1 metric reveals that in every individual test case, both MMAgent and Agent S frameworks consistently expose sensitive information, with a 100% leakage rate across all samples. This indicates that conventional approaches inherently transmit complete GUI data, including passwords and other confidential elements, without any local filtering mechanism. In contrast, MMPro exhibits a substantially lower leakage rate of 10% at the individual sample level, suggesting that its local perception and HDL abstraction successfully obfuscate or eliminate sensitive content in the majority of cases. The aggregate Leak@10 metric further confirms this trend, showing that MMPro reduces overall information exposure to 40% compared to the universal leakage observed in traditional frameworks. These findings quantitatively validate MMPro's architectural advantage in implementing

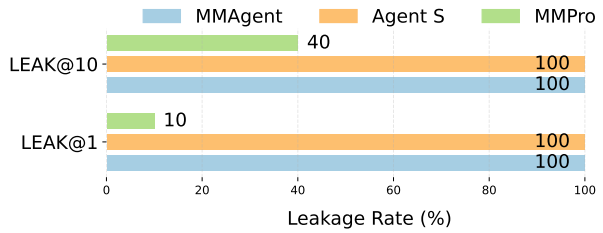

Figure 7: Information Leakage Comparison

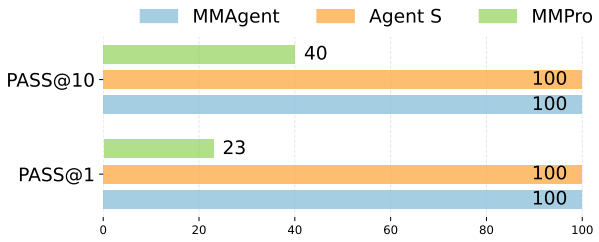

Figure 8: Comparison of success rates of man-in-the-middle.

the principle of minimal disclosure, where the framework intentionally limits the transmission of identifiable personal data to the cloud while maintaining functional utility.

Man-in-the-middle. To evaluate man-in-the-middle attack resilience, we constructed ten distinct attack samples, each replacing legitimate operations with malicious commands, covering diverse exploitation vectors. The attack mechanism involved intercepting and altering API responses: for traditional frameworks, entire command strings were substituted (e.g., changing "ls -l" to "rm -rf /"), while MMPro faced more sophisticated manipulations targeting both reasoning traces and intermediate suggestions to test its localized decision-making robustness. In the results, Pass@1 measures the success rate of individual attack samples across trials, reflecting per-sample vulnerability, while Pass@10 represents the overall attack success rate across all samples, indicating the framework's comprehensive susceptibility.

As shown in Figure 8, MMPro has a significant security advantage over traditional agent frameworks in mitigating man-in-the-middle attacks. Under controlled conditions with ten distinct man-in-the-middle attack samples, each substituting benign operations with hazardous commands, MMPro exhibited markedly lower vulnerability compared to baseline methods. The traditional frameworks in the experiment showed complete vulnerability due to their inherent design flaw of directly executing unverified cloud API outputs. However, MMPro showed significantly improved robustness with a 23% per-sample success rate (pass@1) and a 40% total success rate (pass@10). This performance gap highlights the powerful defense mechanism of MMPro.

The performance gap underscores MMPro's core innovation: decoupling cloud-assisted reasoning from localized decision-making. By delegating final execution authority to the end-side model, MMPro minimizes direct trust in cloud outputs. Even when attackers tamper with intermediate steps, the local model's inherent bias toward safe operations disrupts attack propagation. Nevertheless, MMPro's 60% failure rate for attackers represents a substantial improvement over traditional frameworks, validating its design for high-stakes automation environments.

6 Discussion

MMPro does not require additional training but uses existing multimodal features and structured representations to enhance adaptability, significantly reduce training costs, and improve generalization capabilities, which is superior to models that require fine-tuning or data expansion. Its design of perception-thinking-execution decoupling can be easily integrated into other intelligent agent frameworks, such as the Worker module of Agent S [2] or the expert model of UI-Tars [30] Desktop. This training-free approach benefits sensitive domains like healthcare by enabling modular upgrades without full retraining.

The optimization of current GUI agents should not only focus on the input dimension, as these methods may introduce noise and increase the computational burden. On the contrary, MMPro uses modular decoupling and the concept of "design first" to focus limited computing power on key reasoning links with structured representation. This architectural innovation demonstrates excellent compatibility with model performance enhancement, showing better generalization ability and wider applicability than adding more external knowledge. Experiments show that lightweight end-side models can achieve performance close to that of large cloud models, verifying the optimization path of "high-quality design is better than brute force input".

Limitations and Future Work. Although MMPro excels in privacy protection and task execution, it still has some limitations. First, the parsing ability of HDL is limited by the standardization of the UI. For dynamic or unstructured interfaces, such as games and custom controls, the accuracy of semantic extraction may decrease, resulting in deviations in subsequent steps. Secondly, although modular design improves security, it also introduces additional communication and computing overhead, especially in the cross-modal conversion from perception to thinking and cloud-local collaboration, which may increase system latency. However, we believe that this "time for security" trade-off [16] is reasonable in scenarios with high privacy requirements, such as medical care and finance, where users are usually more concerned about data security rather than millisecond-level responses. In the future, efficiency and security can be balanced by optimizing the dynamic adaptability of HDL and parallelization between modules. Another important direction is to deeply integrate MMPro with privacy reasoning technology. By integrating privacy-enhancing methods designed for LLM [37], the availability and compliance of the system in high-risk scenarios can be significantly improved.

7 Conclusion

In this work, we introduce MMPro, a new multimodal agent designed for GUI interactions that can run without additional training. By leveraging Hybrid Description Language (HDL) and structured representations, MMPro achieves strong task success rates while maintaining strong privacy properties through on-device execution. Compared to existing GUI agents and visual language models, MMPro achieves a unique balance between adaptability, accuracy, and privacy. This makes it a foundation for more efficient, privacy-aware, and adaptable GUI automation, paving the way for further development of multimodal AI systems.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant U2336206 and 62472398, and by Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education (No.KLCS20240207).

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219* (2024).

[2] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. 2024. Agent s: An open agentic framework that uses computers like a human. *arXiv preprint arXiv:2410.08164* (2024).

[3] Anthropic. 2024. Claude 3 Model Card. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf Accessed: 2024-03-04.

[4] Anthropic. 2024. Claude 3.5 Sonnet Released. https://www.anthropic.com/news/claude-3-5-sonnet Accessed: 2024-06-20.

[5] Anthropic. 2025. Claude 3.7 Sonnet Released. https://www.anthropic.com/news/claude-3-7-sonnet Accessed: 2025-04-02.

[6] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. arXiv:2308.12966 [cs.CV] https://arxiv.org/abs/2308.12966

[7] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, et al. 2024. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264* (2024).

[8] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. Seeclick: Harnessing gui grounding for advanced visual gui agents. *arXiv preprint arXiv:2401.10935* (2024).

[9] Lauren L Cloutman. 2013. Interaction between dorsal and ventral processing streams: where, when and how? *Brain and language* 127, 2 (2013), 251–263.

[10] DeepSeek-AI and Daya Guo et.al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv.org/abs/2501.12948

[11] Barry J Dickson. 2002. Molecular mechanisms of axon guidance. *Science* 298, 5600 (2002), 1959–1964.

[12] Erez Freud, David C Plaut, and Marlene Behrmann. 2016. 'What'is happening in the dorsal visual pathway. *Trends in cognitive sciences* 20, 10 (2016), 773–784.

[13] Difei Gao, Siyuan Hu, Zechen Bai, Qinghong Lin, and Mike Zheng Shou. 2024. AssistEditor: Multi-Agent Collaboration for GUI Workflow Automation in Video Creation. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 11255–11257.

[14] Zhiqi Ge, Hongzhe Huang, Mingze Zhou, Juncheng Li, Guoming Wang, Siliang Tang, and Yueting Zhuang. 2024. Worldgpt: Empowering llm as multimodal world model. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 7346–7355.

[15] Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2023. A real-world webagent with planning, long context understanding, and program synthesis. *arXiv preprint arXiv:2307.12856* (2023).

[16] Michael Hilton, Nicholas Nelson, Timothy Tunnell, Darko Marinov, and Danny Dig. 2017. Trade-offs in continuous integration: assurance, security, and flexibility. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 197–207.

[17] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. Cogagent: A visual language model for gui agents. 14281–14290.

[18] Drew A Hudson and Christopher D Manning. 2019. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 6700–6709.

[19] Ole Isacson and Terrence Deacon. 1997. Neural transplantation studies reveal the brain's capacity for continuous reconstruction. *Trends in neurosciences* 20, 10 (1997), 477–482.

[20] Robert JK Jacob. 1996. Human-computer interaction: input devices. *ACM Computing Surveys (CSUR)* 28, 1 (1996), 177–179.

[21] Chengyou Jia, Minnan Luo, Zhuohang Dang, Qiushi Sun, Fangzhi Xu, Junlin Hu, Tianbao Xie, and Zhiyong Wu. 2024. AgentStore: Scalable Integration of Heterogeneous Agents As Specialized Generalist Computer Assistant. arXiv:2410.18603 [cs.AI] https://arxiv.org/abs/2410.18603

[22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246* (2024).

[23] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*. PMLR, 19730–19742.

[24] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916.

[25] Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Iong, Jiadai Sun, Jiaqi Wang, et al. 2024. Autoglm: Autonomous foundation agents for guis. *arXiv preprint arXiv:2411.00820* (2024).

[26] Chuwei Luo, Yufan Shen, Zhaoqing Zhu, Qi Zheng, Zhi Yu, and Cong Yao. 2024. Layoutllm: Layout instruction tuning with large language models for document understanding. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 15630–15640.

[27] OpenAI. 2023. GPT-4V(ision) System Card. https://openai.com/research/gpt-4v-system-card. Accessed: 2023-09-25.

[28] OpenAI. 2025. GPT-4o System Card. https://openai.com/index/gpt-4o-system-card/ Accessed: 2025-04-02.

[29] Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. 2024. Agent q: Advanced reasoning and learning for autonomous ai agents. *arXiv preprint arXiv:2408.07199* (2024).

[30] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326 [cs.AI] https://arxiv.org/abs/2501.12326

[31] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. 2024. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573* (2024).

[32] Yucheng Shi, Wenhao Yu, Wenlin Yao, Wenhu Chen, and Ninghao Liu. 2025. Towards Trustworthy GUI Agents: A Survey. *arXiv preprint arXiv:2503.23434* (2025).

[33] John D Sinden, Helen Hodges, and Jeffrey A Gray. 1995. Neural transplantation and recovery of cognitive function. *Behavioral and Brain Sciences* 18, 1 (1995), 10–35.

[34] Esther T Stoeckli. 2018. Understanding axon guidance: are we nearly there yet? *Development* 145, 10 (2018), dev151415.

[35] Gemini Team and Petko Georgiev et.al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530 [cs.CL] https://arxiv.org/abs/2403.05530

[36] Qwen Team. 2025. QwQ-32B: Embracing the Power of Reinforcement Learning. https://qwenlm.github.io/blog/qwq-32b/

[37] Meng Tong, Kejiang Chen, Jie Zhang, Yuang Qi, Weiming Zhang, Nenghai Yu, Tianwei Zhang, and Zhikun Zhang. 2025. InferDPT: Privacy-preserving Inference for Black-box Large Language Models. *IEEE Transactions on Dependable and Secure Computing* (2025).

[38] Li Wang, Zechen Bai, Yonghua Zhang, and Hongtao Lu. 2020. Show, recall, and tell: Image captioning with recall mechanism. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 12176–12183.

[39] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2024. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191* (2024).

[40] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. 2024. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. *arXiv preprint arXiv:2409.12191* (2024).

[41] Yabing Wang, Le Wang, Qiang Zhou, Zhibin Wang, Hao Li, Gang Hua, and Wei Tang. 2024. Multimodal llm enhanced cross-lingual cross-modal retrieval. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 8296–8305.

[42] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Jing Hua Toh, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems* 37 (2024), 52040–52094.

[43] Dizhan Xue, Shengsheng Qian, and Changsheng Xu. 2024. Few-Shot Multimodal Explanation for Visual Question Answering. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 1875–1884.

[44] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. 2024. Large language model-brained gui agents: A survey. *arXiv preprint arXiv:2411.18279* (2024).

[45] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592* (2023).